M1：模型与工具循环

行为规格、状态机与详细设计

M1：模型与工具循环
- 背景与目标
- 行为规格
  - M1-S01：构造与上下文
  - M1-S02：prompt
  - M1-S03：continue
  - M1-S04：模型流与消息提交
  - M1-S05：终止原因
  - M1-S06：工具执行
  - M1-S07：批量调度和顺序
  - M1-S08：steer 与 followUp
  - M1-S09：取消
  - M1-S10：订阅与运行结果
- 公共数据结构
  - Agent API 与钩子类型
  - 事件信封
- 详细设计
  - 状态机与不变量
  - 队列设计
  - 工具表与并行批
  - 事件与错误边界
  - 假模型设计
- 验收轨迹
- 边界示例与设计推演
  - 两个只读工具反序完成
  - 队列边界
  - 正常运行和基础设施失败的区别
  - 单轮预算计数
- 备选方案、风险与交付顺序

M1：模型与工具循环

属性	内容
状态	设计提案 1.0
依赖	总览的错误合同与预算
交付	`@local/core`、demo 模型、`--print`
验收	M1-A01 至 M1-A12；完全离线

背景与目标

模型可以返回最终文字，也可以要求调用工具。循环需要把工具结果加入上下文再请求模型，同时允许用户取消或补充指令。本里程碑把这些控制规则从供应商和界面中分离出来。

目标是一个确定性的内存运行时：给定相同模型脚本、工具和外部操作，产生相同领域事件和最终消息。非目标是会话落盘、资源发现、真实文件工具和终端渲染。工具调用和消息内容保持文本与 JSON 范围；暂不处理图片和私有推理内容。

行为规格

M1-S01：构造与上下文

`new Agent(options)` 创建 idle 实例。`options` 必须有模型适配器、模型描述和 systemPrompt；工具默认空，maxTurns 默认 32，队列模式默认 `one`。配置无效立即抛 `INVALID_CONFIG`，不调用模型。

初始消息只通过 `replaceContext` 注入。

`replaceContext({systemPrompt,messages,tools,model})` 仅允许 idle，整体校验后一次替换；任何一项无效不修改旧上下文。工具名须匹配 `[A-Za-z0-9_-]{1,64}`，唯一且参数 schema 为对象。消息 ID 全局唯一，tool result 引用的 call 必须存在且没有已有结果。已有 assistant 的调用若尚缺结果，则拒绝；M2 应先修补后再注入。

`state` 返回快照，外部改变其中数组或消息不得改变内部状态。systemPrompt 是独立配置，不伪装成用户消息。工具声明作为模型请求的独立字段发送，不通过修改历史 system 消息宣布变化。

M1-S02：prompt

`prompt(text: string): Promise<RunResult>` 仅允许 idle。空白输入拒绝 `INVALID_INPUT`；繁忙拒绝 `BUSY`；两者都不入队、不产生 run 事件。合法输入保留原字符串，创建 user 消息并启动 run。Promise 在 `agent_end` 和所有关键订阅处理完成后结算。

run 至少执行一次模型请求。最多执行 maxTurns 次；若第 maxTurns 轮仍需要继续，则保存该轮工具结果后以 `RUN_LIMIT` 结束，不能再发第 maxTurns+1 次请求。若末轮在执行工具前已取消，仍走取消规则。

M1-S03：continue

`continue(): Promise<RunResult>` 仅允许 idle、transcript 非空且工具调用均已配对；否则 `BUSY` 或 `INVALID_INPUT`。它不创建新的 user 消息，直接使用当前上下文启动 run。已完成的 assistant 也允许继续，由模型决定补充内容。恢复时不得把它解释为“重放旧工具”。

M1-S04：模型流与消息提交

每次请求包含当前 systemPrompt、transcript 的快照、工具声明和 model。适配器返回 `AsyncIterable<ModelChunk>`，只允许若干 `text_delta` 后跟恰好一个 `final`，final 必须是最后一个 chunk。工具参数只从 final 的完整 JSON 读取；流式半成品不能执行。

`message_start` 在请求前创建 assistant ID 时发出；每个 delta 更新当前显示缓冲并发 `message_update`；final 文本必须与所有 delta 拼接一致，没有 delta 时允许仅 final 提供文本。流正常结束却没有 final、出现两个 final、final 后仍有内容、delta 与 final 不一致，均为 `MODEL_PROTOCOL`。适配器抛错归一成 `MODEL_ERROR`，AbortError 在 signal 已取消时归为 `ABORTED`。

模型不负责消息 ID、timestamp、runId，均由运行时生成。失败时用已收到的文本生成 `stopReason:error` 的 assistant；取消时生成 `stopReason:aborted`。部分流中的工具调用一律丢弃。已提交的消息不会被后续 delta 修改。

M1-S05：终止原因

assistant stopReason	工具调用	行为
`stop`	必须空	保存最终消息，检查 follow-up；没有则完成
`toolUse`	至少一个，callId 不重复	保存消息，执行工具批，再决定继续
`length`	可以有，但不执行	把调用全部标记 `TOOL_BLOCKED`，以失败结束，不请求下一轮
`error`	清空未执行调用	保存错误消息，失败结束
`aborted`	清空未执行调用	保存取消消息，取消结束

`stop` 携带调用、`toolUse` 无调用、重复 callId 是协议错误。在校验 final 时不接受整条非法调用列表，转成无调用的错误 assistant，保证 transcript 不产生无法配对的重复 ID。

M1-S06：工具执行

每条调用依次经过：查找工具 → 解析 JSON 参数 → schema 校验 → before 钩子 → execute → after 钩子 → 结果归一化。schema 禁止额外字段，不做字符串转数字等隐式转换。参数无效不调用钩子或 execute；before 拒绝时不调用 execute。未知工具、非法参数、拒绝、超时和执行抛错均变成一个 `isError:true` 的 tool result，循环可把错误交给模型修正。

`after` 仅对实际调用过 execute 的结果运行，包括 execute 失败；它可以返回替换的 content/details/isError/terminate，但不能改变 callId 和 toolName。after 抛错生成 `TOOL_FAILED`，原执行结果放在 details 中，不重试 execute。before 抛错生成 `TOOL_BLOCKED`。

每个已提交 call 恰好有一个终态 tool result，含相同 callId。更新回调不是终态，execute settle 后的更新丢弃。空文本结果合法。工具返回无法 JSON 序列化的 details 是 `TOOL_FAILED`，不能破坏日志序列化。

M1-S07：批量调度和顺序

工具声明带 `effect:read|write|external`。默认 read 可以并行；任何调用对应 write/external、未知工具或指定串行策略时，整批按源顺序执行。MCP 默认 external，bash 属于 external。先做整批校验与 before 钩子，再开始 execute；钩子按调用源顺序串行运行。

并行批中，execution start 按源顺序，execution update/end 可按实际时间交错。tool result 的 message_start/message_end 必须按 assistant 中调用的源顺序提交，即使后一个先完成。每个结果在其 message_end 监听器处理完成后才算加入已提交历史。

若非空批的每个最终结果均 `terminate:true`，本轮之后结束，不取 follow-up。不满足该条件则继续模型调用；一个失败结果本身不终止整个 run。

M1-S08：steer 与 followUp

`steer(text)` 和 `followUp(text)` 只接受非空字符串，立即返回队列项 ID；繁忙和 idle 都可入队。两队列各自 FIFO、最多 100 项，超限 `INVALID_INPUT` 且队列不变。它们此时不创建 transcript 消息、不落盘。

- steer 在每次模型请求前、前一轮全部工具结果提交之后取出。首轮也检查。
- follow-up 只在 assistant 为 stop、没有工具且无待处理 steer 时取出。
- 模式 `one` 每次取一项，`all` 取当前全部。每个项独立成为一条 user 消息，保持顺序。
- stop 时若 steer 非空，先取 steer 并开始下一轮；follow-up 留待之后。若两队列为空则结束。
- abort、运行错误、RUN_LIMIT 和整批 terminate 不消费剩余项。
- idle 入队不自动启动 run，需显式 prompt 或 continue。
- `clearQueues()` 返回并移除全部未消费项；已转成消息的项不能撤回。

M1-S09：取消

`abort()` 幂等。idle 时无事发生；活动时 abort 当前 run 的 controller。取消后不启动新模型请求、不开始新的工具 execute。已完成结果仍提交；已开始工具必须接收 signal 并在约定时间结束。未开始调用生成 `ABORTED` 结果，确保成对。并行中的工具可能在取消后报告已完成副作用，不能把它改写成“未执行”。

若模型请求正在流式返回，则提交一个 aborted assistant。若取消发生在已完成 assistant 与工具之间，不再制造第二条 assistant，改为为未执行调用补取消结果。若最后结果已提交且 run 正在收尾，abort 不改变既定完成状态。

不响应 signal 的工具在 toolTimeoutMs 到达时结算 `TIMEOUT`；其进程/资源清理由工具实现负责。核心不宣称 Promise.race 能停止外部副作用。`waitForIdle()` 只有收尾和关键写入结束后才返回。

M1-S10：订阅与运行结果

`subscribe(listener,{critical?:boolean})` 返回幂等 unsubscribe。监听器按注册顺序 await，每次分发开始复制监听器列表；分发过程中添加/删除只影响下一事件。不允许关键监听器 await 当前 run 的 prompt/waitForIdle，否则形成循环等待。

普通监听器抛错：发到独立 diagnostic sink，继续后续监听器。关键监听器只供 Session 使用，抛错立即停止事件后续分发和新副作用，abort 正在运行任务，RunResult 为 failed/PERSISTENCE，prompt 拒绝。此异常路径无法承诺 `agent_end` 已持久化，但必须在 finally 将 Core 变回 idle 并结清 waiters。不能为了凑事件序列继续调用已坏掉的存储。

正常/业务失败/取消均恰好一个 `agent_start` 与 `agent_end`。`RunResult` 的 messages 是本次新提交消息，不是全部历史；turns 是实际发出的模型请求数。

公共数据结构

以下类型定义公共边界。示例省略 import；所有 `Json`、`Failure` 和 ID 类型应在本包导出，不依赖其他包。

```
type Json = null | boolean | number | string | Json[] | { [k: string]: Json };
type Failure = { code: string; message: string; details?: Json };
type ToolCall = { callId: string; name: string; arguments: unknown };
type MessageBase = { id: string; timestamp: number };
type UserMessage = MessageBase & { role: "user"; text: string };
type AssistantMessage = MessageBase & {
  role: "assistant";
  text: string;
  calls: ToolCall[];
  stopReason: "stop" | "toolUse" | "length" | "error" | "aborted";
  usage?: { inputTokens: number; outputTokens: number };
  error?: Failure;
};
type ToolResultMessage = MessageBase & {
  role: "toolResult";
  callId: string;
  toolName: string;
```

```
  content: string;
  isError: boolean;
  error?: Failure;
  details?: Json;
};
type AgentMessage = UserMessage | AssistantMessage | ToolResultMessage;
type Model = { id: string; provider: string; contextWindow: number };
type ToolDeclaration = {
  name: string; description: string; inputSchema: Json;
};
type ModelRequest = {
  systemPrompt: string;
  messages: readonly AgentMessage[];
  tools: readonly ToolDeclaration[];
  model: Model;
};
type ModelFinal = Omit<AssistantMessage, "id" | "timestamp" | "role">;
type ModelChunk =
  | { type: "text_delta"; text: string }
  | { type: "final"; message: ModelFinal };
interface ModelAdapter {
  stream(request: ModelRequest, signal: AbortSignal): AsyncIterable<ModelChunk>;
}
```

Json 的运行时校验还须拒绝 NaN/Infinity、undefined、BigInt、函数、类实例和循环引用；不能依赖 JSON.stringify 静默丢字段或把数值改为 null。

`inputSchema` 必须符合配置选定的 JSON Schema 子集。基线支持 object/properties/required/additionalProperties、string、number、integer、boolean、array/items、enum；MCP schema 超出子集时注册失败并给诊断，不能假装已经校验。允许使用 Ajv 扩大支持面，但版本与 schema 方言要固定在锁文件和说明里。

```
type ToolOutcome = {
  content: string;
  isError?: boolean;
  error?: Failure;
  details?: Json;
  terminate?: boolean;
};
interface AgentTool {
  name: string;
  description: string;
  inputSchema: Json;
  effect: "read" | "write" | "external";
  execute(args: unknown, ctx: {
    callId: string;
    signal: AbortSignal;
    update(content: string): void;
  }): Promise<ToolOutcome>;
}
type QueueItem = { id: string; text: string };
type Phase = "idle" | "preparing" | "streaming" | "executing" | "stopping";
type RunResult = {
  runId: string;
  status: "completed" | "failed" | "aborted";
  turns: number;
  messages: AgentMessage[];
  error?: Failure;
};
type AgentSnapshot = {
  phase: Phase;
  runId: string | null;
  messages: readonly AgentMessage[];
  tools: readonly ToolDeclaration[];
```

```
  queued: { steering: number; followUp: number };
};
```

Agent API 与钩子类型

```
type BeforeContext = {
  runId: string; callId: string; toolName: string; args: unknown;
};
type BeforeHook = (ctx: Readonly<BeforeContext>, signal: AbortSignal) =>
  Promise<void | { block: true; reason?: string; terminate?: boolean }>;
type AfterContext = BeforeContext & { outcome: Readonly<ToolOutcome> };
type AfterHook = (ctx: Readonly<AfterContext>, signal: AbortSignal) =>
  Promise<void | Partial<ToolOutcome>>;
type ContextReplacement = {
  systemPrompt: string;
  messages: readonly AgentMessage[];
  tools: readonly AgentTool[];
  model: Model;
};
type AgentOptions = {
  adapter: ModelAdapter;
  systemPrompt: string;
  model: Model;
  tools?: readonly AgentTool[];
  maxTurns?: number;
  toolTimeoutMs?: number;
  queueMode?: { steering: "one" | "all"; followUp: "one" | "all" };
  toolPolicy?: "auto" | "sequential";
  before?: BeforeHook;
  after?: AfterHook;
  clock?: () => number;
  newId?: () => string;
  diagnostic?: (error: Failure) => void;
};
interface AgentPort {
  readonly state: AgentSnapshot;
  prompt(text: string, options?: { runId?: string }): Promise<RunResult>;
  continue(options?: { runId?: string }): Promise<RunResult>;
  replaceContext(context: ContextReplacement): void;
  steer(text: string): string;
  followUp(text: string): string;
  clearQueues(): { steering: QueueItem[]; followUp: QueueItem[] };
  abort(): void;
  waitForIdle(): Promise<void>;
  subscribe(listener: (event: AgentEvent) => void | Promise<void>,
    options?: { critical?: boolean }): () => void;
}
```

`AgentEvent` 按下表定义为以 type 判别的联合，各 variant 都带事件信封。Gateway 可生成 runId 传入 options，让接受响应与第一条事件关联；ID 必须合法且在该 Agent 生命周期唯一，重复拒绝 INVALID_INPUT。省略则由 Core 生成。runId 与消息 ID 使用同一注入工厂，但不能混用命名空间。

clock/newId 只影响可观测标识，不得改变调度。toolPolicy=auto 按 effect 决定并发；sequential 强制串行。before/after 缺省为空链，Gateway 在组合根将扩展责任链作为单个函数注入。此类型面向人手动实现，具体 class 私有字段由实现者决定。

事件信封

每个事件都有 `{runId,seq,timestamp}`；seq 从 1 开始在同一 run 单调递增。只在存在 turn 时带 turnId。message_update 只发当前累计 text，不暴露可变 message 对象。

type	额外字段	含义
`agent_start`	无	run 已独占运行槽
`turn_start`	turnId	即将准备该次模型请求
`message_start`	messageId、role	开始一条消息
`message_update`	messageId、text	assistant 当前累计文字
`message_end`	message	消息已完整；关键监听器在此持久化
`tool_execution_start`	callId、toolName	该调用进入实际执行；被拒绝的不发
`tool_execution_update`	callId、content	显示用中间结果
`tool_execution_end`	callId、outcome	该调用的执行或拒绝已经结算；每 call 一次
`turn_end`	turnId、assistantId、resultMessageIds	这一模型请求及工具批已结清
`agent_end`	result	运行终态，所有消息已提交

user 消息在 agent_start 后、首个 turn_start 前提交；steer 在该轮 turn_start 后、模型 message_start 前提交。没有工具的 turn 也有 turn_end。取消发生在模型请求开始前时可以没有 turn；不要伪造请求计数。execution end 事件不代表已落盘，持久化依赖 tool result 的 message_end。

详细设计

状态机与不变量

当前状态	条件或动作	下一状态
idle	合法 prompt/continue	preparing
preparing	预算和取消检查通过	streaming
streaming	完整工具批提交	executing
executing	结果全部提交，需继续	preparing
streaming	消费 steer/follow-up	preparing
preparing/streaming/executing	完成、取消或错误	stopping
stopping	事件、日志和清理结清	idle

状态机（State）用于把 busy、取消和配置修改的合法时机变成显式转移，不需要为五个状态各建一个类。Agent 负责入口、运行锁和队列；loop 负责轮次推进；tool-runner 负责单个调用。不要让工具回调调用 prompt；它只会遇到 BUSY。

不变量：最多一个 active controller；每次请求捕获不可变快照；完整 assistant 先于工具执行提交；每 call 最多 execute 一次且一个 result；正常结束时没有未配对 call；进入 idle 前 active Promise 必须结算。

队列设计

使用 `{items:QueueItem[],head:number}`，dequeue 递增 head，已消费比例超过一半时压缩数组。enqueue/dequeue 摊销 O(1)，取 k 项 O(k)。禁止反复 `shift` 使大量入队成为平方复杂度。队列项只有在进入 transcript 时才分配 messageId；这让 clearQueue 的撤回语义清楚。

Queue 模式是 Strategy：`takeOne` 与 `takeAll` 改变消费策略，不复制循环。ModelAdapter 也是 Strategy：真实模型和脚本模型实现同一协议，循环不判断 provider 字符串。

工具表与并行批

注册表用 `Map<string,AgentTool>`；模型请求只拿 ToolDeclaration。开始 run 时捕获表，后续 reload 不能改当前批。先将 call 转成固定位置的 task，任务完成时写 `outcomes[index]`。发执行事件按实际完成顺序，发结果消息时按 index 迭代。一次批的额外内存 O(k + 输出字节)，不能以 completion order 重排历史。

串行队列拒绝和 execute 失败都被包装成 settled outcome，不能让一次 Promise 拒绝跳过余下调用的终态结果。abort 检查放在 execute 前，并在 await 返回后再次检查是否应继续请求模型。

事件与错误边界

观察者（Observer）把存储与显示从 Core 解耦。关键订阅提供写入屏障，普通订阅提供 UI；这比所有监听器 fire-and-forget 更容易保证“先记录意图、后执行”。代价是慢监听器会拖慢循环，因此 UI 应只更新内存并调度绘制，不在监听器中做长 I/O。

只在模型边界和工具边界归一化业务错误。不要用一层全局 catch 把存储错误也变成工具结果。finally 必须清理 controller、恢复 idle 并结清 waiters；错误诊断本身抛错时只降级 stderr，不递归分发。

假模型设计

脚本包含每次请求的 chunks、可选 deferred gate 和可选 throw。每调用消耗一个脚本项；耗尽是 MODEL_ERROR。测试记录 requests 的深拷贝，检查第二次请求确实含工具结果。取消通过 signal 释放 gate 并产生 AbortError。demo 脚本可以固定输出，但必须在文案中标明 demo。

验收轨迹

例：用户“查看版本”，模型第一轮调用 readVersion，工具返回 `1.0`，第二轮回复“版本 1.0”。忽略 IDs 和 timestamps 后，关键序列必须是：

```
agent_start
message_start(user), message_end(user)
turn_start
message_start(assistant), message_update*, message_end(assistant/toolUse)
tool_execution_start, tool_execution_end
message_start(toolResult), message_end(toolResult)
turn_end
turn_start
message_start(assistant), message_update*, message_end(assistant/stop)
turn_end
agent_end(completed)
```

`*` 表示零个或多个。tool_execution_end 后，Session 尚可能写入失败，因此不能用它作为恢复的依据。

编号	设置与操作	必须断言
M1-A01	单轮纯文本 prompt	一个 run/turn；最后状态 idle；输入只入历史一次
M1-A02	上述两轮工具轨迹	第二请求含匹配 callId 的结果；事件满足轨迹
M1-A03	活动时再 prompt/replaceContext	BUSY；旧运行快照和队列不变
M1-A04	未知工具、非法参数、before 拒绝	execute=0；每 call 一个错误结果；模型可继续
M1-A05	两个 read；让第二个先结束	execution_end 为 2,1；result 消息为 1,2
M1-A06	read、write、read	三个 execute 无重叠，顺序 1,2,3
M1-A07	toolUse 批结果 terminate 混合/全真	混合继续；全真终止；空批不误判
M1-A08	排两条 steer、一条 followUp	one/all 数量准确；steer 先消费；clear 返回剩余项
M1-A09	streaming 中取消，再次 abort	一个 aborted assistant；无工具；一个终态；队列保留
M1-A10	工具批中取消	已完成结果保留；未启动调用补 ABORTED；无下一模型请求
M1-A11	缺 final、双 final、length 带调用、达到 maxTurns	对应失败；无越界工具/模型请求；结果配对
M1-A12	普通/关键监听器抛错，unsubscribe，延迟 agent_end	普通隔离；关键失败停止；waitForIdle 等待收尾

并发验证用 deferred barrier 记录“是否已开始”，不比较毫秒差。额外测试应覆盖队列上限、消息深拷贝和工具 after 失败，不要求与实现私有字段逐一对应。

边界示例与设计推演

两个只读工具反序完成

assistant 的调用顺序是 A、B。A 等一个 gate，B 立即返回：B 可先发 execution_end，但此时 transcript 不能先插 B result。等 A 完成并发 A end 后，依次提交 result(A)、result(B)，每条等待关键监听器。第二次模型请求看到的末尾始终是 assistant(A,B)、result(A)、result(B)。并行只是执行优化，不改变模型协议的稳定顺序。

若 A 的结果写盘失败，B 虽已执行成功也不能继续请求模型。Session faulted；重开可能补 A/B 为结果未知，不能根据 UI 曾显示 B 成功就重建一个未落盘的结果。

队列边界

假设 steer=[S1,S2]，followUp=[F1]，one 模式。首轮前消费 S1；首轮返回工具调用，工具完成后下一请求前消费 S2；次轮返回 stop，若没有新 steer，消费 F1 再请求；第三轮 stop 才结束。不能把 F1 插在两条工具结果之间，也不能把 S2 丢给已发出的第一轮模型请求。

队列项在边界取出后立即变成 user 消息并提交。若随后模型失败，这些消息仍是历史，clearQueues 不应再次返回它们。

正常运行和基础设施失败的区别

失败点	transcript	Promise / RunResult	继续执行
prompt 空白	不变	拒绝 INVALID_INPUT	可重新提交
模型网络错误	新增错误 assistant	resolve failed/ MODEL_ERROR	本 run 停止，之后可新 run
工具 schema 错	新增错误 tool result	交模型修正，最终结果依后续	可再一轮
关键监听写失败	仅已确认提交的前缀可信	拒绝 PERSISTENCE	Session faulted，不可重用
普通显示监听错	消息不变	diagnostic，不改 run 终态	继续

模型报错消息只有 text/error，不能带未执行 calls。长度限制带调用时则保留原 assistant 和一组明确拒绝结果，因为这条 final 已是结构合法、内容完整的消息，只是不能继续执行。

单轮预算计数

maxTurns=1：纯文本 stop 可以成功；toolUse 可执行这一批工具并提交结果，但需要第二次请求时变为 RUN_LIMIT。不要在看到工具调用时直接跳过整个已许可批，也不要把工具数量当 turn 数。业务方若要限制工具数，应未来新增独立预算，不能改变 maxTurns 定义。

备选方案、风险与交付顺序

把模型和工具混在 `prompt()` 一个函数中最容易起步，但状态与取消难以验证；选定分层循环。AsyncIterable 仅用在模型边界，对外仍是 Promise + 订阅；若让 UI 消费生成器并控制推进，慢界面容易改变工具执行。

先定义消息和脚本模型，再实现单轮纯文本、串行工具、批量调度、队列、取消，最后接 `--print`。每次只扩大一条可验证轨迹。不要提前加入磁盘。M2 会依赖 message_end 的关键订阅语义，所以该边界必须先锁定。